

UNIT-3 DATABASE APPLICATION DESIGN AND DEVELOPMENT

EMBEDED SQL:

Embedded SQL is a method of inserting inline SQL statements or queries into the code of a programming language, which is known as a host language. Because the host language cannot parse SQL, the inserted SQL is parsed by an embedded SQL preprocessor. Embedded SQL is a robust and convenient method of combining the computing power of a programming language with SQL's specialized data management and manipulation capabilities.

Embedded SQL is not supported by all relational database management systems (RDBMS). Oracle DB and PostgreSQL provide embedded SQL support. MySQL, Sybase and SQL Server 2008 do not, although support was provided by earlier versions of SQL Server (2000 and 2005).

The C programming language is commonly used for embedded SQL implementation. For example, a commercial bank's information system (IS) has a front-end user interface created in the C language, and the IS interfaces with a back-end Oracle DB database. One of the front-end interface modules allows quick viewing and commission calculation for sales agents during specified periods. An inefficient approach to handling this process would be to store each commission value in a database table. However, a more effective solution is to calculate and return commission values based on unique user requests on specified dates. The application accomplishes this by embedding a SQL query within the C code, as follows:

```
SELECT      0.2*SALE_AMOUNT      FROM      TOTAL_SALES      WHERE  
SALE_DATE='MM/DD/YYYY' AND AGENT_NO=xx
```

Dynamic SQL:

Dynamic SQL is a programming technique that could be used to write SQL queries during runtime. Dynamic SQL could be used to create general and flexible SQL queries.

Syntax for dynamic SQL is to make it string as below:

'SELECT statement';

To run a dynamic SQL statement, run the stored procedure **sp_executesql** as shown below :

EXEC sp_executesql N'SELECT statement';

Use prefix N with the sp_executesql to use dynamic SQL as a Unicode string.

Steps to use Dynamic SQL:

1. Declare two variables, @var1 for holding the name of the table and @var 2 for holding the dynamic SQL :
2. DECLARE
3. @var1 NVARCHAR(MAX),
@var2 NVARCHAR(MAX);
4. Set the value of the @var1 variable to table_name :
SET @var1 = N'table_name';
5. Create the dynamic SQL by adding the SELECT statement to the table name parameter :
6. SET @var2= N'SELECT *

FROM ' + @var1;

7. Run the sp_executesql stored procedure by using the @var2 parameter :
EXEC sp_executesql @var2;

Example –

SELECT *

from geek;

Table – Geek

ID	NAME	CITY
1	Khushi	Jaipur
2	Neha	Noida
3	Meera	Delhi

Using Dynamic SQL :

DECLARE

@tab NVARCHAR(128),

@st NVARCHAR(MAX);

SET @tab = N'geektable';

SET @st = N'SELECT *

FROM ' + @tab;

EXEC sp_executesql @st;

Table – Geek

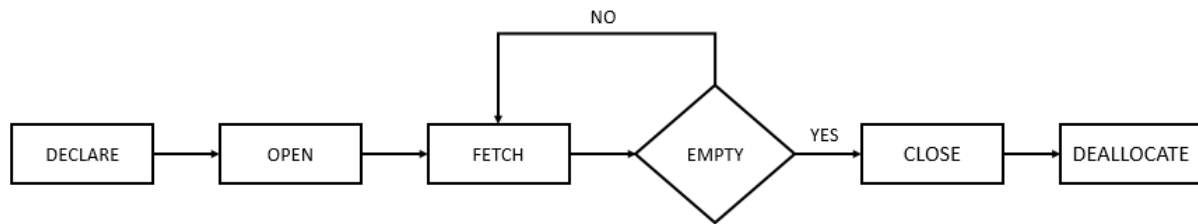
ID	NAME	CITY
1	Khushi	Jaipur
2	Neha	Noida
3	Meera	Delhi

Database cursor:

A database cursor is an object that enables traversal over the rows of a result set. It allows you to process individual row returned by a query.

SQL Server cursor life cycle:

These are steps for using a cursor:



First, declare a cursor.

```
DECLARE cursor_name CURSOR
```

```
FOR select_statement;
```

To declare a cursor, you specify its name after the DECLARE keyword with the CURSOR data type and provide a SELECT statement that defines the result set for the cursor.

Next, open and populate the cursor by executing the SELECT statement:

```
OPEN cursor_name;
```

Then, fetch a row from the cursor into one or more variables:

```
FETCH NEXT FROM cursor INTO variable_list;
```

SQL Server provides the @@FETCHSTATUS function that returns the status of the last cursor FETCH statement executed against the cursor; If @@FETCHSTATUS returns 0, meaning the FETCH statement was successful. You can use the WHILE statement to fetch all rows from the cursor as shown in the following code:

```
WHILE @@FETCH_STATUS = 0
```

```
BEGIN
```

```
    FETCH NEXT FROM cursor_name;
```

```
END;
```

After that, close the cursor:

```
CLOSE cursor_name;
```

Finally, deallocate the cursor:

```
DEALLOCATE cursor_name;
```

SQL Server cursor example

We'll use the production.products table from the sample database to show you how to use a cursor:

First, declare two variables to hold product name and list price, and a cursor to hold the result of a query that retrieves product name and list price from the production.products table:

```

DECLARE
    @product_name VARCHAR(MAX),
    @list_price DECIMAL;
DECLARE cursor_product CURSOR
FOR SELECT
    product_name,
    list_price
FROM
    production.products;

```

Next, open the cursor:

```
OPEN cursor_product;
```

Then, fetch each row from the cursor and print out the product name and list price:

```

FETCH NEXT FROM cursor_product INTO
    @product_name,
    @list_price;
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @product_name + CAST(@list_price AS varchar);
    FETCH NEXT FROM cursor_product INTO
        @product_name,
        @list_price;
END;

```

After that, close the cursor:

```
CLOSE cursor_product;
```

Finally, deallocate the cursor to release it.

```
DEALLOCATE cursor_product;
```

The following code snippets put everything together:

```

DECLARE
    @product_name VARCHAR(MAX),
    @list_price DECIMAL;
DECLARE cursor_product CURSOR
FOR SELECT
    product_name,
    list_price
FROM
    production.products;
OPEN cursor_product;
FETCH NEXT FROM cursor_product INTO
    @product_name,
    @list_price;
WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @product_name + CAST(@list_price AS varchar);

```

```
FETCH NEXT FROM cursor_product INTO
    @product_name,
    @list_price;
END;
CLOSE cursor_product;
DEALLOCATE cursor_product;
```

Procedure:

Before proceeding with procedures and cursors, you will need to know a bit about PL/SQL which is a block-structured language that enables developers like you to combine the power of SQL with procedural statements. But you will not learn in a conventional way, you will learn it as you go along and as required. So if you have an SQL query and you want to execute it multiple times. Procedures are one of the solutions for it. Often procedures are called in this context because they remain stored and get triggered upon a particular action or a series of actions. Procedures are also referred to as Process.

Writing procedures:

The general syntax for writing a procedure is as follows:

```
CREATE PROCEDURE procedure_name
AS
sql_statement
GO;
```

Please note that these syntaxes applies to almost any RDBMS be it Oracle, be it PostgreSQL or be it MySQL. After you have created a procedure you will have to execute it. Following is the syntax for that.

```
EXEC procedure_name;
```

You will write a procedure named SelectAllCustomers which will select all the customers from Customers.

```
CREATE PROCEDURE SelectAllCustomers
AS
SELECT * FROM Customers
GO;
```

Execute SelectAllCustomers by:

```
EXEC SelectAllCustomers;
```

Procedures can be stand-alone blocks of statements as well which makes them independent of any tables unlike the previous one. The following example creates a simple procedure that displays the string 'Hello World!' as output when executed.

```
CREATE PROCEDURE welcome
AS
BEGIN
dbms_output.put_line('Hello World!');
END;
```

There are two ways of executing a stand-alone procedure.

- Using the EXEC keyword
- Calling the name of the procedure from a PL/SQL block

The above procedure named 'welcome' can be called with the EXEC keyword as:

```
EXEC welcome;
```

A procedure can be replaced as well. You just need to add REPLACE keyword when you are creating the procedure. This will replace the already existing procedure (if) otherwise will create a fresh one.

```
CREATE OR REPLACE PROCEDURE welcome
```

```
AS
```

```
BEGIN
```

```
dbms_output.put_line('Hello World!');
```

```
END;
```

Deleting a stored procedure is no big deal:

```
DROP PROCEDURE procedure-name;
```

You will write a stored procedure that selects Customers from a particular City from the table:

```
CREATE PROCEDURE SelectAllCustomers @City n varchar(30)
```

```
AS
```

```
SELECT * FROM Customers WHERE City = @City
```

```
GO;
```

Let's dissect the common principles here:

You wrote the first @City and defined its type and size as one of the parameters which will be given when the procedure will be executed.

The second @City is assigned to condition variable City which is nothing but a column in the Customers table.

The procedure is executed as :

```
EXEC SelectAllCustomers City = "London";
```

AQJ CENTRE